```
14  static ushort semarray[2]={1,1};
15
16  key=ftok(".","h");
17  semid=semget(key,2,IPC_CREAT|0744);
18  arg.array=semarray;
19  semctl(semid,0,SETALL,arg);
20  for(i=0;i<2;i++){
21  ret=semctl(semid,i,GETVAL,0);
22  printf("sem %d=%d\n",i,ret);
23  }
24  }
```

We'll now create Program 4 and 5 to create the 'dead lock' situation as can be understood from Table 1.

```
1  /*SemDeadLock1.c---This program demonstrates the improper
2  * sequence of locking leading to deadlock situation*/
3
4  #include <stdio.h>
5  #include <sys/sem.h>
6
7  main(){
8  int key,semid;
9  key = ftok(".","h");
10 semid = semget(key,2,0);
11 struct sembuf s[2]={{0,-1,0|SEM_UNDO},{1,-1,0|SEM_UNDO}};
12
13 printf("locking semaphore1\n");
14 semop(semid,&s[0],1);
15 printf("Inside critical section :----semaphore1\n");
16 getchar();
17
18 printf("\n locking semaphore2\n");
19 semop(semid,&s[1],1);
20 printf("Inside critical section :----semaphore2\n");
21 getchar();
22
23 s[0].sem_op = 1;
24 semop(semid,&s[0],1);
25 printf("semaphore1 unlocked\n");
26
27 s[1].sem_op = 1 ;
28 semop(semid,&s[1],1);
29 printf("semaphore 2 unlocked\n");
30 }
```

```
1  /*SemDeadLock2.c---This program demonstrates the improper
2  * sequence of locking leading to deadlock situation*/
3
4  #include <stdio.h>
5  #include <sys/sem.h>
6
7  main(){
8  int key,semid;
9  key = ftok(".","h");
10 semid = semget(key,2,0);
11 struct sembuf s[2]={{0,-1,0|SEM_UNDO},{1,-1,0|SEM_UNDO}};
12
13 printf("locking semaphore2\n");
14 semop(semid,&s[1],1);
15 printf("Inside critical section :----semaphore2\n");
16 getchar();
17
18 printf("\n locking semaphore1\n");
19 semop(semid,&s[0],1);
20 printf("Inside critical section :----semaphore1\n");
21 getchar();
22
23 s[1].sem_op = 1;
24 semop(semid,&s[1],1);
25 printf("semaphore2 unlocked\n");
26
27 s[0].sem_op = 1 ;
28 semop(semid,&s[0],1);
29 printf("semaphore1 unlocked\n");
30 }
```

Program 4	Program 5
Lock S1	Lock S2
getchar ( )	getchar ( )
Lock S2	Lock S1
getchar ( )	getchar ( )
Unlock S1	Unlock S2
Unlock S2	Unlock S1

Table 1

Implementation: If the first process locks the first critical section using the first semaphore (S1) and later locks the second critical section using the second semaphore (S2), then unlock S1 and S2. The second process locks the first critical section using S2 and the second critical section is locked by S1. In this scenario, both the processes wait indefinitely. This leads to the classical race condition. This is depicted in Program 4 and Program 5. So, if you know how to create a 'dead lock' situation, then obviously you know how to avoid the unwanted dead lock. To avoid dead lock, don't use more than one lock inside a critical section. But if you want to do that, you should preserve the same sequence of locking in all the processes.

Acknowledgement

The authors would like to thank Mr V Jayasurya for his critical reading of this manuscript.

By: Shobana V and Dr B Thangaraju

The authors work with Talent Transformation, Wipro Technologies, Bangalore. They can be reached at shobana.venkatesan@wipro.com and balat.raju@wipro.com, respectively.